

AVET: A Provenance-Aware Venue Emulator Trace Dataset for Event Markets

Maksym Nechepurenko*

August 2026

Abstract

Venue integrations are often tested through endpoint mocks, isolated lifecycle fixtures, or final-balance assertions. Those artifacts omit the ordered evidence needed to reproduce how a venue moved from market data to order acceptance, matching, settlement, close, resolution, redemption, or operational failure. This paper introduces the Axient Venue Emulator Trace Dataset (AVET), a deterministic and provenance-aware format for event-venue emulator traces, and reports the public AVET v0.2.0 corpus. An AVET trace is a self-contained ordered event stream with source provenance, logical and observed clocks, causation identifiers, typed payloads, publication boundaries, and layered cryptographic identities. The specification distinguishes synthetic, rights-cleared derived, raw-permitted, hash-only, and private-not-distributed source classes. AVET v0.2.0 contains 60 fully synthetic source-generated traces from an immutable emulator release: twelve each in BOOK, ORDER, SETTLEMENT, LIFECYCLE, and FAULT. The corpus contains 240 events, fourteen event types, seven sequence families, and a 40/10/10 core, stress, and negative split. All traces pass strict offline schema and semantic validation; twelve malformed controls are rejected; 240 complete-trace identity mutations are detected; and Python and Node.js agree on 12,000 recursive key-order checks. Event-stream hashes and trace digests are publicly recomputable, while scenario-bundle and terminal-state hashes are explicitly labelled source commitments. AVET is not empirical evidence about a live venue and does not estimate latency, liquidity, availability, fill quality, or settlement reliability. Its contribution is a reusable trace object, source-anchored synthetic corpus, explicit data-rights boundary, exact replay tooling, and publication protocol for venue adapters, audit tools, and future empirical calibration.

*Founder and Director of Research, ForesightFlow, the Research Department of Devnull FZCO, Dubai, United Arab Emirates. Email: maksym@devnull.ae.

Keywords: event markets; venue emulator; trace dataset; event sourcing; settlement evidence; provenance; reproducibility; fault injection.

ACM CCS: Information systems—Data provenance; Software and its engineering—Software testing and debugging.

JEL Classification: C63, G13, G14, G23.

1 Motivation and scope

An event-market integration is a temporal system. A snapshot arrives, an order is accepted or rejected, matches may occur in pieces, settlement can lag matching, the market may close early, an oracle may dispute or void the event, and redemption may fail after resolution. A final balance cannot reveal whether the integration handled those transitions correctly.

Event sourcing and logical clocks provide a natural representation of ordered histories [Lamport, 1978, Schneider, 1990]. Data documentation and provenance standards emphasize that a dataset must state how observations were produced and what uses they support [Wilkinson et al., 2016, Gebru et al., 2021, World Wide Web Consortium, 2013]. AVET combines these ideas with deterministic emulator execution and cryptographic trace identities.

The dataset is venue-neutral and reusable. It does not contain Axient credit pools, leverage admission, provider accounting, reserves, debt, fee routing, or loss waterfalls. Those belong to the consuming protocol. AVET represents the external venue side of the boundary: market data, orders, matching, settlement, close, resolution, redemption, and faults.

The concise revision presents the formal trace object, provenance policy, canonical 60-trace corpus, replay and validation evidence, and publication governance. Extended event registries and trace catalogues remain in the earlier arXiv version history and public release.

Proposition 1.1 (Emulation–execution non-equivalence). *A deterministic emulator trace is evidence that a consuming system handled a specified venue path. It is not evidence that a live venue will exhibit the same latency, liquidity, matching, settlement, or failure distribution.*

2 Trace object and provenance

Definition 2.1 (Canonical event envelope). *An event envelope contains at least*

$$E = (id, type, payload, logical\ time, observed\ time, causation, correlation, source, hash). \quad (1)$$

Payloads are typed by event kind. Logical time controls deterministic replay; observed time records ingestion or emission when available.

Definition 2.2 (AVET trace). *A trace is*

$$T = (I, P, M, S_0, (E_1, \dots, E_n), S_n, Q, H), \quad (2)$$

where I is trace identity metadata, P provenance, M market definition, S_0 initial venue state, (E_k) the ordered event stream, S_n terminal venue oracle, Q quality and semantic assertions, and H layered identities.

A self-contained trace has enough information to validate event order, typed payloads, causal links, terminal state, and publication boundary without access to private operational systems. It need not redistribute the raw source payload.

Definition 2.3 (Provenance classes). *Each source is classified as one of:*

- (a) *SYNTHETIC*;
- (b) *DERIVED_PERMITTED*;
- (c) *RAW_PERMITTED*;
- (d) *HASH_ONLY*; or
- (e) *PRIVATE_NOT_DISTRIBUTED*.

Proposition 2.4 (Provenance without redistribution). *A public trace can preserve source identity and transformation lineage through content hashes, retrieval time, schema version, and derivation metadata even when the raw third-party payload cannot be redistributed.*

This distinction is essential. Public availability of an API does not automatically grant permission to republish its raw data. AVET therefore treats data rights as a per-source and, when needed, per-trace decision.

3 Domain model and clocks

A venue state contains markets, outcomes, books, orders, matches, settlements, resolutions, redemptions, balances, capabilities, and incidents. The base trace model supports binary and multi-outcome markets without assuming a particular matching engine.

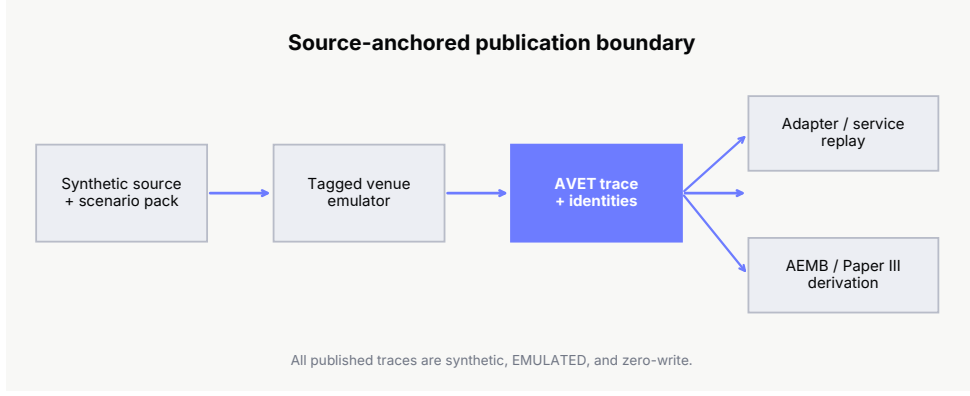


Figure 1: AVET publication boundary between source data, the reusable emulator, trace generation, and consuming systems.

Relevant clocks include:

$$t_s = \text{source timestamp}, \quad t_i = \text{ingestion timestamp}, \quad (3)$$

$$t_l = \text{logical emulator time}, \quad t_o = \text{order time}, \quad (4)$$

$$t_m = \text{match time}, \quad t_{set} = \text{settlement time}, \quad (5)$$

$$t_c^{sched} = \text{scheduled close}, \quad t_c^{act} = \text{actual close}, \quad (6)$$

$$t_f = \text{final resolution}, \quad t_r = \text{redemption}. \quad (7)$$

A trace may contain both deterministic logical time and observed wall-clock time. The former supports exact replay; the latter supports empirical analysis only when it originates from a permitted real observation.

3.1 Market, order, and settlement semantics

BOOK traces distinguish a full snapshot from an incremental delta. A delta is admissible only when sequence numbers, parent hash, and market identity match the current state. A gap invalidates quote admission until a resnapshot restores continuity. Staleness is measured against a declared policy clock rather than inferred from whether an endpoint responded.

ORDER traces represent submit, accept, reject, cancel, expire, and partial-fill states. An accepted order is not a match, and a match is not a settlement. Each fill carries quantity, price, fee, maker/taker role when available, and causation links to the order and market state used by the emulator.

SETTLEMENT traces distinguish pending, confirmed, replay, conflict, and failure. A settlement event includes an evidence identifier and content hash. Replaying the same content is state-inert; reusing an external identifier with different content opens a

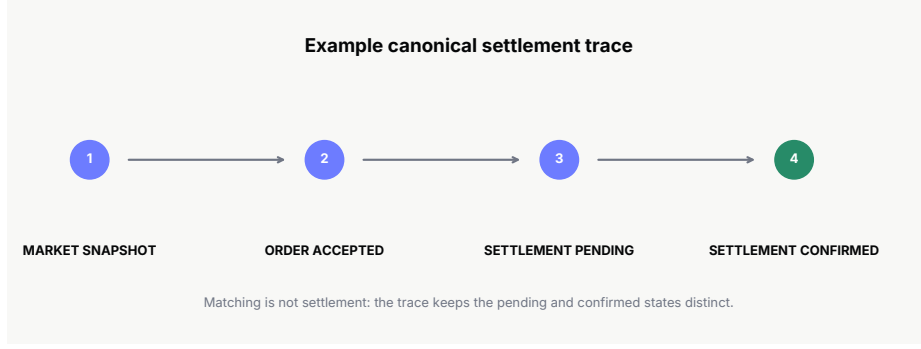


Figure 2: Illustrative order-to-settlement trace with separate logical, match, settlement, and finality clocks.

conflict rather than overwriting history. These semantics are useful to consuming credit protocols but remain venue-side trace facts, not debt-accounting rules.

LIFECYCLE traces distinguish scheduled close, actual close, resolution, void, and redemption. The trace can therefore represent a market that becomes non-tradable before its published close or resolves before redemption is complete. FAULT traces cover process restart, sequence loss, telemetry outage, capability expiry, and recovery without introducing a successful financial outcome by default.

The event model covers seven sequence families:

- (1) snapshot and incremental book reconstruction;
- (2) order submit, accept/reject, and cancel;
- (3) match and partial-fill progression;
- (4) settlement pending, confirmed, replay, conflict, and failure;
- (5) scheduled/actual close and venue suspension;
- (6) resolution, void, and redemption; and
- (7) operational fault, recovery, and capability change.

Sequence completeness requires that every event has a unique ID, known type, monotone logical position, valid causal predecessor when required, and a payload consistent with the state transition. Source completeness records whether source payload, normalized representation, or only a hash is available. Terminal completeness means the final oracle is sufficient to verify all registered semantic assertions.

4 Identity and canonicalization

Canonical JSON follows a versioned RFC 8785-style serialization [Rundgren et al., 2020]. Hashes use SHA-256 [National Institute of Standards and Technology, 2015]. AVET

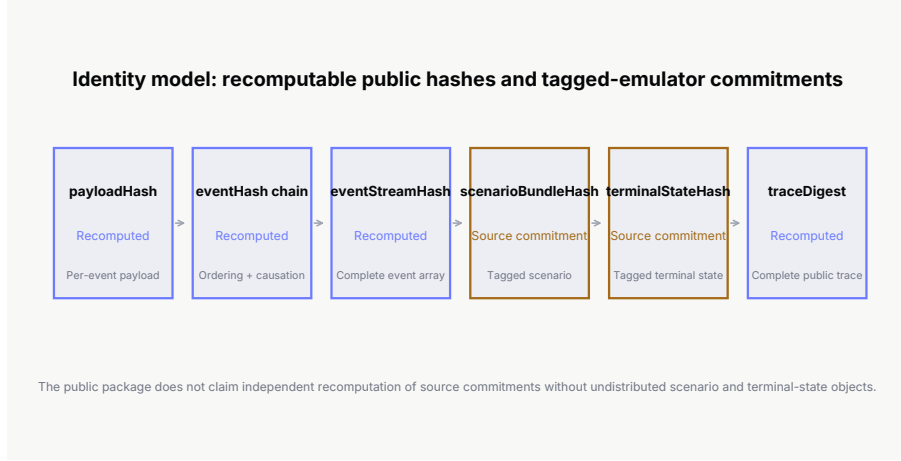


Figure 3: Publicly recomputable trace identities and source-anchored commitments.

separates five identities:

$$h_k^{payload} = \text{SHA256}(\text{Canon}(\text{payload}_k)), \quad (8)$$

$$h_k^{event} = \text{SHA256}(h_{k-1}^{event} \parallel \text{Canon}(E_k)), \quad (9)$$

$$h^{stream} = \text{SHA256}(h_n^{event}), \quad (10)$$

$$h^{terminal} = \text{source commitment to terminal state}, \quad (11)$$

$$h^{trace} = \text{SHA256}(\text{Canon}(T)). \quad (12)$$

Proposition 4.1 (Hash-chain sensitivity). *Deleting, duplicating, reordering, or modifying a semantically included event changes the downstream event hash, event-stream hash, and complete-trace digest except with negligible collision probability.*

Proposition 4.2 (Source commitment is not trace identity). *A scenario-bundle or terminal-state commitment can bind a public trace to an immutable emulator source without being publicly recomputable when the underlying source object is not distributed. Such a commitment must not be described as an independently computed public identity.*

In AVET v0.2.0, payload hashes, event chains, event-stream hashes, and complete-trace digests are publicly recomputable. Scenario-bundle and terminal-state hashes are labelled tagged-emulator source commitments.

5 Canonical corpus

AVET v0.2.0 contains 60 fully synthetic traces generated from the immutable emulator export recorded in the public release manifest. The collections are balanced.

The corpus contains 240 ordered events and fourteen event types. The split is 40 core,

Table 1: AVET v0.2.0 composition.

Collection	Traces	Primary scope
BOOK	12	snapshot, delta, gap, stale book
ORDER	12	submit, reject, cancel, partial fill
SETTLEMENT	12	pending, confirmed, replay, conflict, failure
LIFECYCLE	12	close, suspension, resolution, redemption
FAULT	12	outage, restart, recovery, capability change
Total	60	

10 stress, and 10 negative traces. Negative traces are expected to end in a rejection, quarantine, timeout, or other fail-closed state; they are not failed dataset records.

Every trace records:

$$\begin{aligned} \text{executionMode} &= \text{EMULATED}, & \text{externalWriteCount} &= 0, \\ \text{productionAssetMovement} &= 0. \end{aligned} \tag{13}$$

It also records that raw third-party data, credentials, and user identifiers are absent.

The canonical corpus was reconciled against an earlier independently generated 60-trace collection. All matching identifiers were classified as semantic conflicts because the precursor encoded different trace semantics. The public corpus was regenerated through the tagged emulator rather than silently importing those files. The precursor remains research input, not canonical data.

6 Replay and semantic validation

A public AVET consumer should be able to validate a trace without the original emulator process. The reference replay procedure is intentionally simple.

- R1:** Parse the trace and validate the top-level schema, provenance record, market definition, initial state, event envelopes, quality object, and identity fields.
- R2:** Recompute every payload hash, then replay the chained event hashes in declared logical order.
- R3:** Apply each typed event to a deterministic venue-state reducer. Events with invalid preconditions, missing causal parents, duplicate IDs, or non-monotone logical clocks fail the trace.
- R4:** Compare the reconstructed terminal state with the public terminal oracle when distributed, or verify the declared source commitment when only a commitment is available.

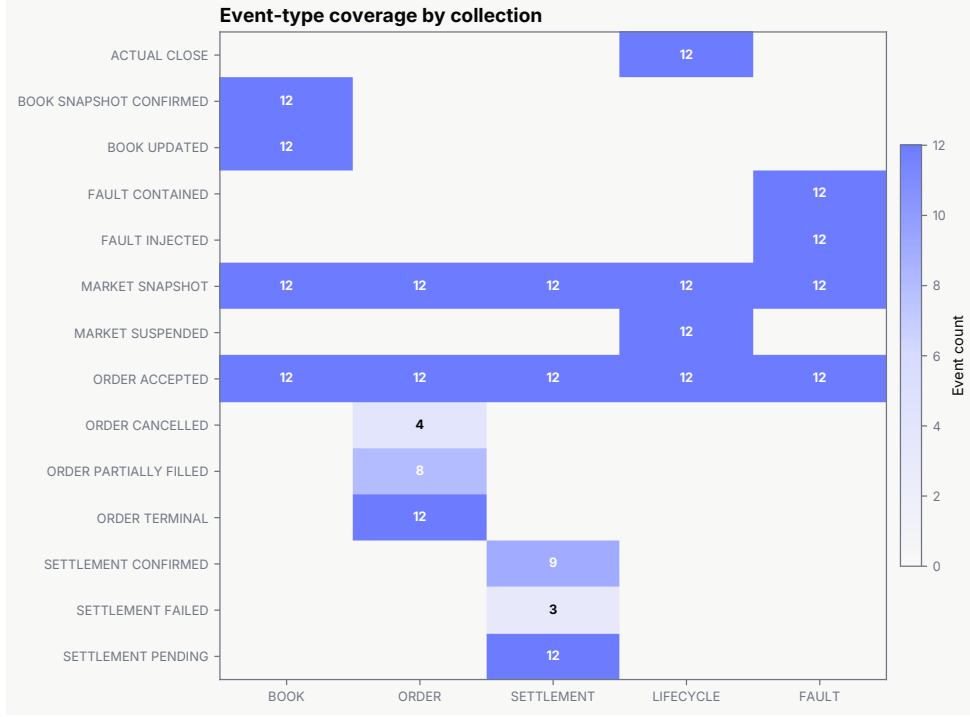


Figure 4: Event-type coverage across BOOK, ORDER, SETTLEMENT, LIFECYCLE, and FAULT collections.

R5: Evaluate semantic assertions, including zero prohibited writes, expected rejection or quarantine, and collection-specific invariants.

R6: Recompute the event-stream hash and complete-trace digest, then compare them with the manifest.

The state reducer is venue-neutral. A book delta changes depth only after sequence and parent validation. An accepted order creates an active order but not a match. A match changes matched quantity but not settlement. A settlement confirmation changes the settlement record and balance projection. Close, resolution, and redemption are independent transitions. Fault and recovery events can change capability or process state without manufacturing a financial success.

Proposition 6.1 (Replay determinism). *For a fixed schema, initial state, ordered event sequence, and deterministic reducer, independent conforming implementations produce the same public terminal oracle and recomputable trace identities.*

The proposition does not apply to undistributed source commitments. Those link the trace to the tagged emulator but require the source object for recomputation.

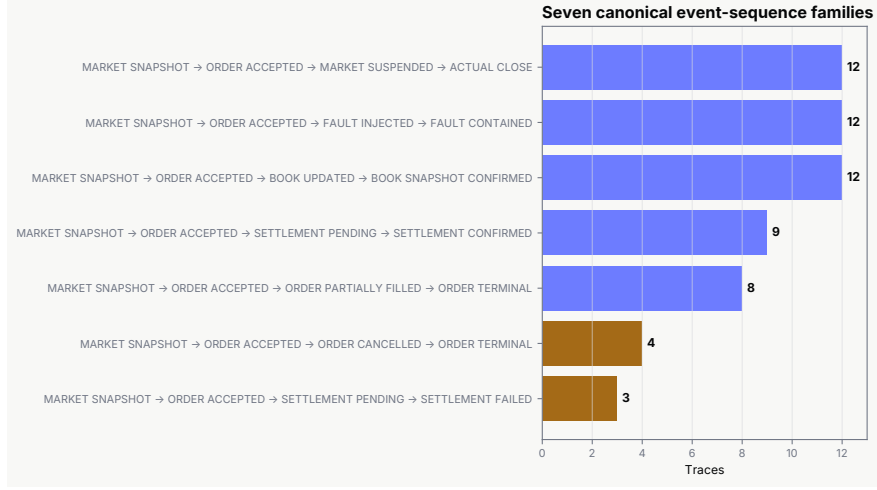


Figure 5: Trace-length distribution by collection and scenario class.

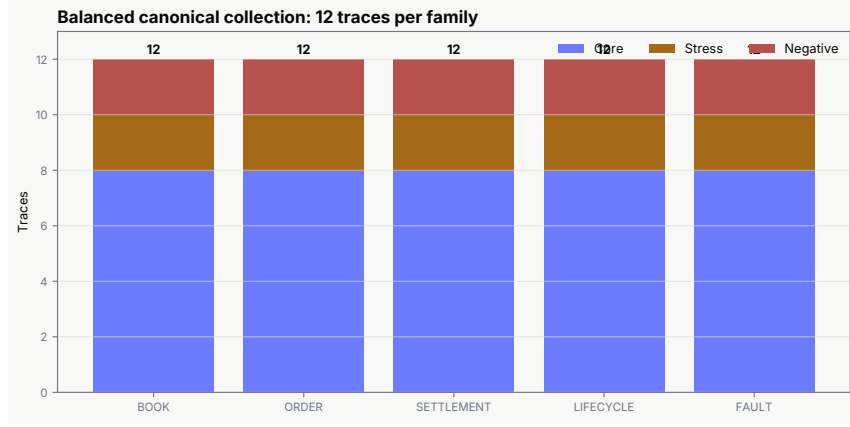


Figure 6: Twelve traces in each of five venue-lifecycle collections.

6.1 Schema strictness

The JSON Schema uses closed objects where silent extension would alter semantics and versioned unions for event payloads. Required fields include source class, execution mode, scenario class, logical order, causation/correlation identifiers, payload hash, boundary counters, and identity declarations. An extension field is permitted only inside an explicitly open metadata object and is included or excluded from identity according to the schema version.

Malformed controls cover missing provenance, unknown event type, duplicate event ID, invalid hash format, broken causation, decreasing logical clock, external-write count above zero, production-asset movement, and incomplete terminal evidence. Semantic validation is stricter than structure: a well-formed settlement confirmation that exceeds matched capacity must still fail.

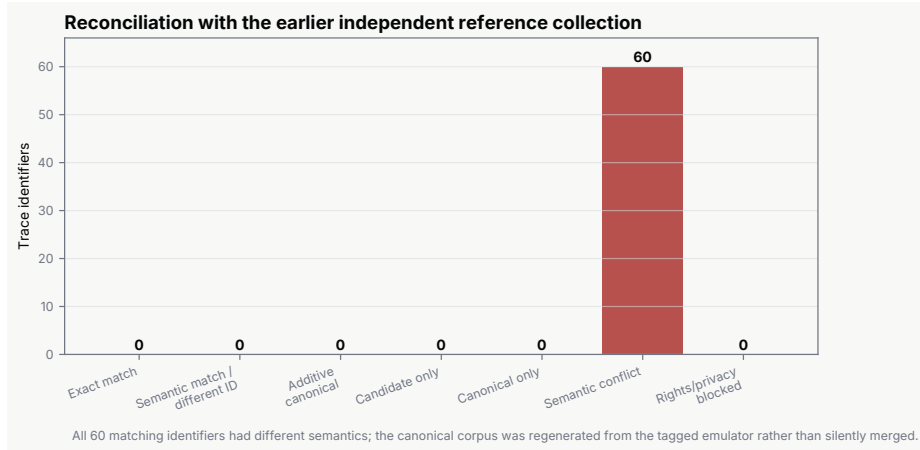


Figure 7: Reconciliation of the canonical tagged-emulator corpus with the earlier independent reference collection. Conflicts are retained as evidence rather than silently overwritten.

6.2 Deterministic generation

For source-generated traces, the release records emulator repository, commit, annotated tag, scenario bundle hash, seed, schema identity, and generator version. Two clean builds must produce byte-identical public packages after excluding only declared nondeterministic archive metadata. If generator output changes, the release receives a new version or a documented reconciliation result; the old package is not overwritten.

7 Validation and reproducibility

The public package includes strict offline JSON Schema validation, semantic checks, malformed controls, identity mutation tests, independent canonicalizers, and a manifest verifier.

Registered results are:

Key-order testing recursively permutes object keys while preserving arrays and numeric/string values. Exact cross-runtime agreement demonstrates canonicalization consistency, not semantic completeness. Mutation testing changes one complete-trace field at a time and verifies that identity changes.

Two clean release builds are byte-identical under the pinned toolchain. The public package includes schema, validators, replay tools, Dataset Card, provenance, licensing, citation metadata, SBOM, and SHA-256 manifests.

Table 2: AVET v0.2.0 validation evidence.

Check	Result
Schema and semantic-valid traces	60 / 60
Malformed controls rejected	12 / 12
Complete-trace identity mutations detected	240 / 240
Python/Node.js recursive key-order checks	12,000 / 12,000
Unique event-stream hashes	60
Unique complete-trace digests	60
Manifest entries verified	all
External writes / production asset movement	0 / 0

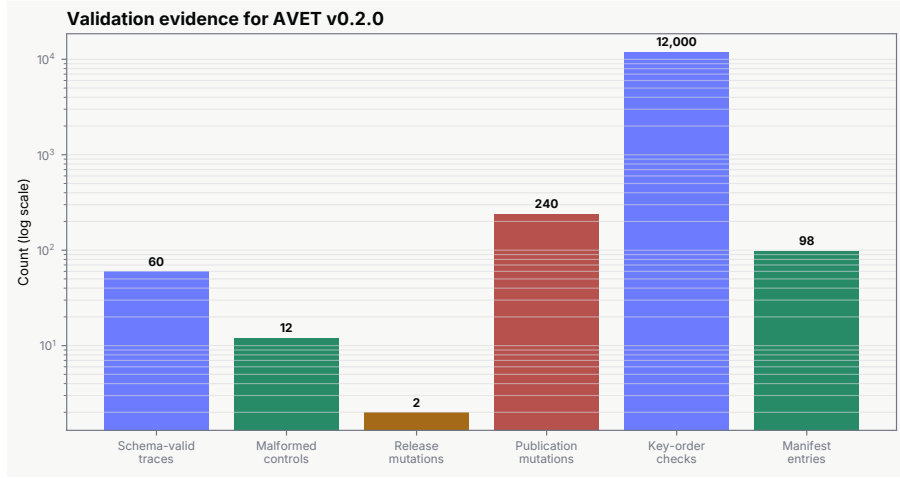


Figure 8: Independent schema, malformed-control, mutation, parity, and manifest evidence.

8 Quality dimensions

AVET reports quality as a vector rather than a single score.

- (1) **Schema validity**: the trace and every event satisfy the versioned structural contract.
- (2) **Sequence completeness**: event order, causal references, and logical clocks are consistent.
- (3) **Source completeness**: the public record states whether raw, derived, or hash-only provenance is available.
- (4) **Terminal completeness**: the terminal oracle supports every registered semantic assertion.
- (5) **Identity integrity**: payload, chain, stream, and complete-trace identities verify.
- (6) **Boundary integrity**: no prohibited raw data, identifiers, credentials, external writes, or production asset movement appear.

A trace may be structurally valid yet unsuitable for a particular empirical question.

Table 3: Evidence supported by source class.

Source class	Permitted interpretation
SYNTHETIC	Deterministic lifecycle and failure handling; no live-frequency inference.
DERIVED PERMITTED	Analysis of published normalized features under source-specific rights.
RAW PERMITTED	Full public replay and independent transformation, subject to the licence.
HASH_ONLY	Provenance linkage and integrity checks; no raw-payload reconstruction.
PRIVATE NOT_DISTRIBUTED	Internal analysis only; a public release may contain aggregate results or commitments.

For example, a fully synthetic settlement trace can test replay logic but cannot estimate settlement latency. A hash-only trace can preserve source linkage while preventing independent reconstruction of the undistributed payload. Consumers must select traces by quality and provenance class rather than treating all records as interchangeable.

9 Use with protocol benchmarks and empirical studies

AVET and AEMB solve different problems. AVET preserves long venue-side histories. AEMB encodes compact protocol obligations with expected terminal states and negative boundaries. A long AVET trace can be analyzed and minimized into one or more AEMB cases, but the original trace should remain available for causal and temporal context.

Typical uses include:

- testing venue adapters and replay engines;
- reproducing match-versus-settlement behavior;
- validating recovery after sequence gaps, restarts, or capability expiry;
- comparing audit and observability tools;
- generating negative controls for event-margin protocols; and
- designing future empirical data collection with explicit provenance.

For empirical calibration, AVET supplies a schema, not measurements. Rights-cleared live or sandbox traces would need source-specific retention, privacy, and redistribution decisions. They should be released as a new dataset version or separate empirical corpus, not mixed silently with synthetic v0.2.0.

10 Governance and limitations

Fully synthetic traces and public outputs are licensed CC BY 4.0. Schemas, generators, validators, and replay tools are Apache-2.0. Axient marks are separately governed. Raw or derived third-party venue traces remain private by default.

Semantic versioning distinguishes incompatible schema changes, additive trace releases, and metadata-only patches. Published files are not silently replaced. A correction receives a new version, manifest, and tombstone or explanation when appropriate.

AVET has several limitations. The corpus is finite and author-designed. It does not prove that all venue failures are represented. A validator can reproduce a shared specification error. Synthetic timestamps and liquidity do not estimate live distributions. Source commitments cannot be recomputed without the undistributed emulator objects. Event traces also do not establish that a consuming financial protocol applies correct debt, reserve, or withdrawal semantics; that is the role of protocol conformance benchmarks and implementation tests.

The dataset is therefore evidence of reproducible trace handling, not venue quality or product safety.

11 Conclusion

AVET makes venue behavior inspectable as an ordered, provenance-aware object rather than a collection of disconnected mocks. The public 60-trace corpus covers books, orders, settlement, lifecycle, and faults under one deterministic emulator identity. Layered hashes detect payload, ordering, terminal, and complete-trace changes; provenance classes separate reproducibility from unauthorized redistribution; and exact replay tooling allows independent implementations to verify the same trace semantics.

The dataset’s value lies in its boundary discipline. It is fully synthetic, makes no live-performance claim, and separates publicly recomputable identities from source commitments. That discipline permits AVET to support venue-adaptor engineering, audit evidence, AEMB case generation, and future empirical calibration without confusing emulation with observation.

12 Compact trace catalogue

The sixty traces are organized as deterministic scenario families rather than isolated examples. Within each collection, four traces cover the normal path, two extend it with partial or delayed progression, two exercise explicit rejection or conflict, two model an

Table 4: Trace-family design.

Collection	Core distinctions	Stress and negative boundaries
BOOK	snapshot, incremental delta, crossed/uncrossed book, resnapshot	sequence gap, stale snapshot, invalid parent hash, liquidity disappearance
ORDER	submit, accept, cancel, partial fill, expiration	unauthorized write, invalid size, duplicate cancel, capability loss
SETTLEMENT	pending, partial, confirmed, redemption-linked	failure, replay, conflicting evidence, excess settlement
LIFECYCLE	scheduled close, actual close, resolution, void, redemption	early suspension, delayed finality, failed redemption, invalid transition
FAULT	worker restart, telemetry recovery, resync, capability refresh	repeated crash, unrecoverable gap, pause, orderly wind-down

operational stress, and two provide negative controls. The exact catalogue is versioned in the public archive; the following table summarizes the intended distinctions.

This design is deliberately balanced, not statistically representative. It gives every major lifecycle family both ordinary and fail-closed examples while keeping release size small enough for exact inspection.

Data and code availability

AVET v0.2.0 is available at <https://github.com/AxientLab/avet/releases/tag/avet-v0.2.0>. DOI information will be added in a later revision.

Disclosure

Automated language and code-assistance tools were used for editorial review, schema construction, synthetic trace generation, validation, consistency checking, and figure production. The author specified and reviewed the dataset purpose, domain model, provenance policy, identity rules, trace catalogue, validation design, interpretation, and final manuscript, and takes responsibility for the work.

References

- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. doi: 10.1145/359545.359563.

- National Institute of Standards and Technology. Secure hash standard (SHS), fips pub 180-4. U.S. Department of Commerce, 2015.
- Anders Rundgren, Bret Jordan, and Samuel Erdtman. RFC 8785: Json canonicalization scheme (JCS). RFC Editor, 2020.
- Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, 1990. doi: 10.1145/98163.98167.
- Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3: 160018, 2016. doi: 10.1038/sdata.2016.18.
- World Wide Web Consortium. PROV-O: The PROV ontology. W3C Recommendation, 2013.